To construct the ground truth for evaluating detection accuracy, we developed explicit labeling guidelines for all 15 multi-language code smells. These guidelines were created to ensure **transparency, reproducibility, and consistency** in the manual identification process. For each smell, we provide:

- a **plain-language definition** to clarify its intent,
- **identification criteria** expressed as observable checks, and
- a **clear example** of code that satisfies the criteria.

By codifying the rules in this format, we minimize subjectivity in labeling and enable independent researchers to replicate our annotation process. Two annotators applied these guidelines independently, and disagreements were resolved through discussion until consensus was reached.

1. Not Handling Exceptions

- **Definition:** JNI methods that may throw exceptions are called without explicit handling (e.g., `ExceptionCheck()`, `ExceptionClear()`, `ThrowNew()`, or try-catch).
- **Identification Criteria:**
    1. Call to methods like `FindClass`, `GetFieldID`, `GetMethodID`, etc.
    2. No exception check or handling after the call.
- **Example:**

```
jclass cls = env->FindClass("com/example/MyClass");
// No exception handling after call
return cls;
```

---

2. Assuming Safe Return Value

- **Definition:** JNI methods return a value that is used or returned without checking for errors.
- **Identification Criteria:**
    1. JNI call assigns result to a variable.
    2. The variable is returned or propagated.
    3. No error check performed before returning.
- **Example:**

```
jclass cls = env->FindClass("com/example/MyClass");
return cls; // Returned without checking null/error
```

---

3. Not Securing Libraries

- **Definition:** A native library is loaded without security wrappers (e.g., `AccessController.doPrivileged`) and without exception handling.

- **Identification Criteria:**
    1. Call to `System.loadLibrary` or similar.
    2. Not enclosed in privileged block or try-catch.
- **Example:**

```
System.loadLibrary("nativeLib");
```

---

4. Hard Coding Libraries

- **Definition:** Library loading uses an absolute path tied to a specific operating system.
- **Identification Criteria:**
    1. Library path includes absolute directories (e.g., `C:/`, `/usr/lib/`).
    2. Path differs by OS without portability handling.
- **Example:**

```
System.load("/usr/lib/nativeLib.so");
```

---

5. Not Using Relative Path

- **Definition:** Library is loaded from an absolute path instead of a relative path.
- **Identification Criteria:**
    1. Call to `System.load`.
    2. Absolute path used instead of relative.
- **Example:**

```
System.load("/home/user/project/lib/nativeLib.so");
```

---

6. Too Much Clustering

- **Definition:** A class declares too many native methods (≥ threshold, default 8), and they are invoked externally.
- **Identification Criteria:**
    1. Count of native methods in a class ≥ threshold.
    2. At least one method invoked outside the class.
- **Example:**

```
public class NativeWrapper {
    public native void m1(); public native void m2(); public native void
m3();
```

```
    public native void m4(); public native void m5(); public native void
m6();
    public native void m7(); public native void m8();
}
OtherClass obj = new OtherClass();
obj.useWrapper(new NativeWrapper());
```

---

7. Too Much Scattering

- **Definition:** A package contains many native classes (≥ threshold), each with few native methods.
- **Identification Criteria:**
    1. Package has more than threshold native classes.
    2. Each class has fewer than MaxMethodsThreshold native methods.
- **Example:**

```
// Package with multiple small native classes
package com.example.nativepkg;
public class A { public native void a1(); }
public class B { public native void b1(); }
public class C { public native void c1(); }
```

---

8. Excessive Inter-language Communication (EILC)

- **Definition:** A native method is called excessively within a class, within loops, or with repeated parameters.
- **Identification Criteria:**
    1. Count of calls to native method exceeds threshold.
    2. OR calls occur inside loops.
    3. OR same parameter passed repeatedly beyond threshold.
- **Example:**

```
for (int i = 0; i < 100; i++) {
    nativeObj.process(i); // excessive loop calls
}
```

---

9. Local References Abuse

- **Definition:** Excessive local references are created without deleting them or increasing capacity.
- **Identification Criteria:**
    1. JNI calls like `NewLocalRef`, `AllocObject`, `NewObject*` used repeatedly.

2. No corresponding `DeleteLocalRef` or `EnsureLocalCapacity`.

- **Example:**

```
for (int i = 0; i < 1000; i++) {
    jobject obj = env->NewObject(cls, mid);
    // No DeleteLocalRef
}
```

---

10. Memory Management Mismatch

- **Definition:** Memory is allocated with JNI “get” functions but never released with the corresponding “release” functions.
- **Identification Criteria:**
  1. Calls to methods like `GetStringUTFChars`, `GetIntArrayElements`.
  2. No corresponding `Release*` call.
- **Example:**

```
const char* str = env->GetStringUTFChars(jstr, NULL);
// Forgot to call ReleaseStringUTFChars
```

---

11. Not Caching Objects

- **Definition:** Field or method IDs are repeatedly looked up instead of being cached.
- **Identification Criteria:**
  1. Repeated calls to `GetFieldID`, `GetMethodID`, etc. for same class/object.
  2. No caching strategy used.
- **Example:**

```
for (int i = 0; i < 100; i++) {
    jmethodID mid = env->GetMethodID(cls, "doWork", "()V");
    env->CallVoidMethod(obj, mid);
}
```

---

12. Excessive Objects

- **Definition:** JNI object fields are accessed repeatedly without setting them or managing usage.
- **Identification Criteria:**
  1. Multiple `Get*Field` calls on same object parameter.
  2. No corresponding `Set*Field`.
- **Example:**

```
jint val1 = env->GetIntField(obj, fid);
jint val2 = env->GetIntField(obj, fid);
// No SetField, repeated gets
```

---

13. Unused Method Implementation

- **Definition:** A native method is declared and implemented but never called.
- **Identification Criteria:**
    1. Declared in Java with `native`.
    2. Implemented in C/C++.
    3. Not invoked anywhere in Java code.
- **Example:**

```
public native void unusedMethod();
// implemented in C, but never called in Java
```

---

14. Unused Method Declaration

- **Definition:** A native method is declared in Java but not implemented in C/C++.
- **Identification Criteria:**
    1. Declared in Java with `native`.
    2. No matching implementation in C/C++.
- **Example:**

```
public native void notImplemented();
// no C/C++ implementation exists
```

---

15. Unused Parameters

- **Definition:** Parameters of native methods are declared but not used in the implementation.
- **Identification Criteria:**
    1. Parameter declared in Java native method.
    2. Implemented in C/C++ method.
    3. Parameter never referenced inside method body.
- **Example:**

```
public native void withUnusedParam(int x);

JNIEXPORT void JNICALL Java_MyClass_withUnusedParam(JNIEnv* env, jobject
obj, jint x) {
    // x is never used
```

```
}
```